Sistema de Totem de Caixa de Supermercado

Bárbara Stella Sena Barbosa, Cecília Scofield Carneiro, Larissa Rafaele Vieira Silva, Pedro Vieira Andrade
Escola de Engenharia
Universidade Federal de Minas Gerais
Belo Horizonte, Brasil
bstellasna@gmail.com, csc.scofield@gmail.com, larissavieira@ufmg.br, pedro@imop.com.br

***Resumo*— Este trabalho apresenta o desenvolvimento de um sistema de autoatendimento e gerenciamento de caixa para supermercado, elaborado em Python como projeto final da disciplina Programação Orientada a Objetos (POO). A aplicação permite o cadastro e a identificação de clientes, a consulta de produtos, a montagem de carrinhos, a simulação de pagamentos e o registro do histórico de compras, por meio da utilização dos principais conceitos estudados, como classes abstratas, herança, polimorfismo e encapsulamento.**

Palavras-chave—Python, POO, Caixa de Supermercado, Autoatendimento

I. INTRODUÇÃO

A automação de processos de atendimento tem se tornado cada vez mais presente em diferentes setores da sociedade, especialmente em estabelecimentos comerciais e serviços que lidam diariamente com um grande volume de consumidores. No setor supermercadista, essa transformação pode ser observada na implementação de terminais de autoatendimento, sistemas de leitura automática de produtos e soluções digitais para gerenciamento das operações de compra. Essas tecnologias permitem que parte das atividades tradicionalmente realizadas por funcionários seja executada diretamente pelos clientes, modificando a dinâmica de atendimento e a organização dos estabelecimentos.

Entre as principais aplicações dessa automação estão os sistemas de *self-checkout*, também conhecidos como caixas de autoatendimento. Por meio deles, o consumidor pode registrar os produtos, acompanhar o valor da compra e realizar o pagamento sem depender integralmente de um operador de caixa. A adoção desses recursos está relacionada à busca por maior praticidade, à redução do tempo de espera e ao aumento da eficiência operacional. No Brasil, a Associação Brasileira de Supermercados (ABRAS) já destaca o self-checkout e outras tecnologias de automação entre as soluções que vêm sendo incorporadas ao setor [1]. Entretanto, para que esses sistemas funcionem adequadamente, é necessário integrar diferentes operações, como identificação de clientes, consulta de produtos, controle de estoque, cálculo de valores e registro de compras.

Nesse contexto, o desenvolvimento de um sistema de caixa automatizado exige mais do que a criação de uma interface para interação com o consumidor. É necessário organizar as informações e estabelecer uma comunicação adequada entre os diferentes componentes responsáveis pelas operações do estabelecimento. O controle de permissões dos funcionários, a atualização das quantidades disponíveis no estoque e o armazenamento do histórico de compras são exemplos de funcionalidades que precisam atuar de maneira integrada para garantir a consistência das informações.

Diante disso, este trabalho apresenta o desenvolvimento de um sistema de totem de caixa de supermercado utilizando a linguagem Python. A aplicação simula o funcionamento de um sistema de autoatendimento, permitindo que clientes cadastrados consultem produtos, adicionem itens ao carrinho, finalizem compras e acompanhem seu histórico. Além disso, contempla uma área destinada aos funcionários, com diferenciação de permissões entre gerente e operador de caixa e funcionalidades relacionadas ao gerenciamento do estoque.

O projeto foi desenvolvido como aplicação prática dos conceitos de Programação Orientada a Objetos, buscando representar os elementos envolvidos no funcionamento de um supermercado por meio de classes e objetos com responsabilidades específicas. Dessa forma, além de simular operações presentes em sistemas reais de autoatendimento, o trabalho possibilita explorar a organização, a reutilização e a manutenção de código por meio dos princípios fundamentais da POO.

II. METODOLOGIA

O desenvolvimento do sistema foi realizado utilizando a linguagem Python e a abordagem de Programação Orientada a Objetos (POO). A modelagem, disponível em [2], foi organizada em classes que representam os principais elementos envolvidos nas operações de um mercado, permitindo distribuir as responsabilidades entre diferentes componentes e estabelecer a interação entre eles. Durante a implementação, foram utilizados os conceitos de abstração, encapsulamento, herança e polimorfismo, além da definição de classes abstratas para estabelecer estruturas comuns e comportamentos obrigatórios.

A. Princípios da Programação Orientada a Objetos

1) Abstração

A abstração consiste em representar elementos de um problema por meio de suas características e comportamentos essenciais, desconsiderando detalhes que não são relevantes para o funcionamento do sistema. No projeto, esse princípio foi aplicado na representação dos principais elementos de um supermercado - clientes, funcionários, produtos, estoque e caixa. Cada classe reúne as informações e operações necessárias para desempenhar sua função, sem que os demais componentes precisem conhecer todos os detalhes de sua implementação.

2) Encapsulamento

O encapsulamento consiste em agrupar atributos e métodos em uma mesma classe e controlar o acesso às informações internas de seus objetos. Seu objetivo é evitar alterações indevidas no estado dos objetos e tornar o código

mais organizado e seguro. No sistema, os atributos de diferentes classes foram definidos como privados, utilizando o prefixo duplo de sublinhado. O acesso a essas informações ocorre por meio de propriedades implementadas com '@property' e, quando necessário, de métodos setters, responsáveis por controlar as alterações. Na classe 'Caixa', por exemplo, o carrinho é mantido como atributo privado, e a propriedade carrinho retorna uma cópia das quantidades armazenadas, evitando que sua estrutura interna seja modificada diretamente por outros componentes.

3) Herança

A herança permite que uma classe derivada reutilize atributos e métodos de outra classe, denominada classe-mãe, possibilitando especializar comportamentos sem repetir toda a implementação. No projeto, esse conceito aparece em duas estruturas principais. A classe 'Produto' é herdada por 'ProdutoUnitario' e 'ProdutoPesavel', que representam produtos vendidos por unidade e por peso, respectivamente. De maneira semelhante, a classe 'Funcionario' é herdada por 'Gerente' e 'OperadorCaixa', que representam diferentes categorias de funcionários do supermercado. Assim, as informações e funcionalidades comuns são definidas nas classes-mãe, enquanto as subclasses implementam suas particularidades.

4) Polimorfismo

O polimorfismo permite que objetos de classes diferentes respondam de maneiras distintas a uma mesma chamada de método. No sistema, esse princípio é observado principalmente no método 'calcular_valor_venda()', implementado pelas classes 'ProdutoUnitario' e 'ProdutoPesavel'. Embora ambas realizem o cálculo do valor de venda, a primeira multiplica o preço pela quantidade de unidades, enquanto a segunda utiliza o peso do produto em quilogramas. Outro exemplo ocorre no método 'obter_permissoes()', implementado pelas classes 'Gerente' e 'OperadorCaixa', que retornam permissões diferentes de acordo com o cargo do funcionário.

5) Classes Abstratas

As classes abstratas estabelecem uma estrutura comum para outras classes e podem definir métodos que obrigatoriamente deverão ser implementados pelas subclasses. Em Python, esse recurso pode ser utilizado por meio da biblioteca 'abc', com a utilização de 'ABC' e do decorador '@abstractmethod'. No projeto, as classes 'Produto' e 'Funcionario' são abstratas. A primeira define o método 'calcular_valor_venda()', enquanto a segunda estabelece o método 'obter_permissoes()'. Dessa forma, as subclasses concretas precisam implementar esses comportamentos para que seus objetos possam ser instanciados.

B. Organização das Classes do Sistema

O projeto foi organizado em seis módulos principais, armazenados na pasta 'src', além de uma pasta 'data', destinada ao armazenamento das informações utilizadas pela aplicação. Essa divisão permite separar as responsabilidades do sistema e facilita a manutenção do código. A seguir, são apresentadas as principais classes desenvolvidas e suas funcionalidades.

1) Classe 'Sistema'

A classe 'Sistema', implementada no módulo 'sistema.py', é responsável por coordenar o funcionamento geral da aplicação e controlar a navegação entre os menus. Seu construtor inicializa os componentes necessários para a execução, incluindo os objetos responsáveis pelo estoque e pelo caixa.

O método 'iniciar()' apresenta o menu inicial e conduz o usuário às diferentes áreas do sistema. A classe também possui métodos específicos para organizar o login de clientes e funcionários, o cadastro de novos clientes e a navegação pelos menus de compras e de gestão. Entre eles, destacam-se '_login_cliente()', '_menu_cliente_principal()', '_menu_funcionario_login()', '_menu_gestao()', '_registrar_cliente()' e '_menu_compras_cliente()'. Além disso, os métodos '_carregar_funcionarios()' e '_carregar_clientes()' recuperam os registros armazenados nos arquivos de texto. Portanto, essa classe atua como ponto de integração entre as funcionalidades implementadas nos demais módulos.

2) Classe 'Cliente'

A classe 'Cliente', presente no módulo 'cliente.py', representa os consumidores cadastrados no sistema. Seus principais atributos são nome, CPF, total gasto e histórico de compras, todos encapsulados. As propriedades permitem consultar as informações do cliente sem expor diretamente seus atributos internos.

O método 'registrar_compra()' adiciona uma nova operação ao histórico e atualiza o valor total gasto pelo cliente. Já 'mostrar_historico()' apresenta as compras realizadas e o valor acumulado. Para garantir a persistência dessas informações, foram implementados os métodos 'salvar_historico()' e 'carregar_historico()', responsáveis por gravar e recuperar os registros em um arquivo de texto individual, identificado pelo CPF do consumidor. O módulo também possui a função 'carregar_clientes_do_txt()', utilizada para recuperar os clientes cadastrados no arquivo geral.

3) Classe 'Funcionario'

A classe 'Funcionario', definida no módulo 'funcionario.py', é abstrata e reúne os atributos comuns aos funcionários do supermercado: nome, identificador, senha e estado de autenticação. Esses atributos são privados e possuem propriedades para consulta e alteração, com validações aplicadas em determinadas operações.

O método 'autenticar()' verifica se a senha informada corresponde à senha cadastrada e, em caso positivo, atualiza o estado de login do funcionário. Além disso, a classe

estabelece o método abstrato ‘obter_permissoes()’, que deve ser implementado pelas subclasses.

A classe ‘OperadorCaixa’ herda de ‘Funcionario’ e implementa o método ‘obter_permissoes()’ com acesso restrito ao processamento de vendas. Já a classe ‘Gerente’ implementa o mesmo método retornando permissões mais amplas, que incluem operações de caixa, alteração do estoque e acesso a relatórios. Essa estrutura permite representar diferentes níveis de acesso por meio de herança e polimorfismo. O módulo também contém a função ‘carregar_funcionarios_do_txt()’, responsável por recuperar os funcionários cadastrados e instanciar a classe correspondente ao cargo informado.

4) Classe ‘Produto’

A classe ‘Produto’, implementada no módulo ‘produtos.py’, é abstrata e representa a estrutura comum dos produtos comercializados. Seus atributos incluem identificador, nome, preço e disponibilidade. O preço pode ser consultado e alterado por meio de uma propriedade com setter, que impede a atribuição de valores menores ou iguais a zero. A disponibilidade também pode ser modificada pelo método ‘atualizar_disponibilidade()’.

A classe define o método abstrato ‘calcular_valor_venda()’, cuja implementação é responsabilidade das subclasses. A classe ‘ProdutoUnitario’ representa os produtos comercializados por unidade e calcula o valor da venda multiplicando o preço pela quantidade informada. Por sua vez, ‘ProdutoPesavel’ representa os produtos comercializados por peso e realiza o cálculo utilizando o preço por quilograma e o peso adquirido. Essa especialização permite que o sistema trate diferentes modalidades de venda utilizando uma estrutura comum.

O módulo também apresenta a função ‘carregar_produtos_do_txt()’, que realiza a leitura dos produtos armazenados no arquivo de estoque e instancia objetos da classe correspondente ao tipo de venda informado.

5) Classe ‘Estoque’

A classe ‘Estoque’, definida no módulo ‘estoque.py’, é responsável pelo gerenciamento dos produtos e de suas respectivas quantidades disponíveis. Entre suas principais funcionalidades estão a busca de produtos pelo identificador, a inclusão e remoção de produtos, a consulta de quantidades e a atualização do estoque.

O método ‘buscar_produto()’ localiza um produto cadastrado, enquanto ‘adicionar_produto()’ e ‘remover_produto()’ permitem modificar os itens registrados. Para controlar as quantidades, são utilizados os métodos ‘consultar_quantidade()’, ‘adicionar_quantidade()’ e ‘retirar_quantidade()’. O método ‘listar_produtos()’ apresenta os produtos disponíveis, e ‘repor_item()’ permite realizar a reposição de mercadorias durante as operações de gestão.

A classe também é responsável pela persistência dos dados do estoque. Para isso, os métodos ‘carregar_estoque()’ e ‘salvar_estoque()’ realizam, respectivamente, a leitura e a gravação das informações no arquivo de texto correspondente. Dessa maneira, as alterações realizadas durante a utilização do sistema podem ser recuperadas posteriormente.

6) Classe ‘Caixa’

A classe ‘Caixa’, implementada no módulo ‘caixa.py’, controla as operações relacionadas à montagem do carrinho e à finalização das compras. Para isso, recebe uma referência ao objeto ‘Estoque’, permitindo consultar os produtos disponíveis e atualizar suas quantidades durante as vendas. O carrinho é armazenado como um atributo privado, associando cada produto à quantidade adicionada.

O método ‘adicionar_item()’ verifica a existência do produto, valida a quantidade informada e verifica se há estoque suficiente. Também considera as diferenças entre produtos unitários e pesáveis, exigindo quantidades inteiras para os primeiros. O método ‘remover_item()’ permite retirar um produto do carrinho, enquanto ‘visualizar_carrinho()’ apresenta os itens selecionados, seus subtotais e o valor total da compra.

O cálculo é realizado pelo método ‘calcular_total()’, que percorre os produtos adicionados e utiliza o método polimórfico ‘calcular_valor_venda()’ de cada objeto para determinar os respectivos subtotais. Por fim, ‘finalizar_compra()’ valida a forma de pagamento, calcula o troco quando necessário, retira do estoque as quantidades adquiridas e registra a operação no histórico do cliente. O método ‘iniciar_atendimento()’ organiza a interação com o usuário durante o processo de compra, apresentando as opções de consulta de produtos, inclusão e remoção de itens, visualização do carrinho e finalização ou cancelamento da operação.

C. Persistência e Organização dos Dados

Para manter as informações utilizadas pelo sistema, foi implementada a leitura e gravação de arquivos de texto na pasta data. O arquivo ‘clientes.txt’ armazena os clientes cadastrados, enquanto ‘funcionarios.txt’ reúne os dados dos funcionários e seus respectivos cargos. O arquivo ‘estoque.txt’ contém os produtos e suas quantidades, e os arquivos individuais de histórico armazenam as compras realizadas por cada cliente.

Essa organização permite que os dados sejam recuperados durante a execução da aplicação e que alterações, como a reposição de produtos e a finalização de compras, sejam registradas para utilizações posteriores. É importante destacar que essa implementação utiliza persistência em arquivos de texto, não um sistema gerenciador de banco de dados relacional.

D. Diagrama UML

O diagrama UML, que pode ser encontrado em [3], apresenta a estrutura das classes desenvolvidas, destacando seus principais atributos, métodos e relacionamentos. Por meio dele, é possível visualizar as relações de herança entre as classes abstratas e suas subclasses, assim como a colaboração entre os componentes responsáveis pelo cadastro, gerenciamento de estoque e processamento das compras. O diagrama complementa a descrição da arquitetura do sistema e facilita a compreensão da distribuição de responsabilidades entre os objetos.

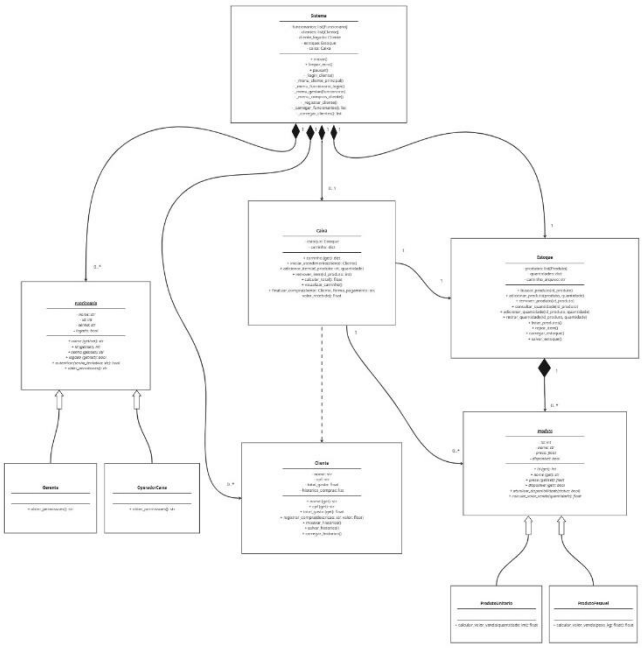


Fig. 1 – Diagrama UML

III. TESTES E RESULTADO

Foram realizados testes manuais por meio da execução dos menus no terminal. As capturas de tela registram os principais cenários utilizados para verificar a interação com o sistema, a validação das entradas, o controle de acesso e a atualização dos dados.



Fig. 2 - Menu principal apresentado ao iniciar a aplicação.

A Fig. 2 apresenta o menu inicial, que direciona o usuário para a área do cliente, a área do funcionário ou a saída. Nos testes de cadastro, o sistema rejeitou o envio de nome e CPF vazios e também identificou a presença de letras no campo destinado ao CPF. Esses resultados demonstram a aplicação de validações antes da gravação do novo cliente.

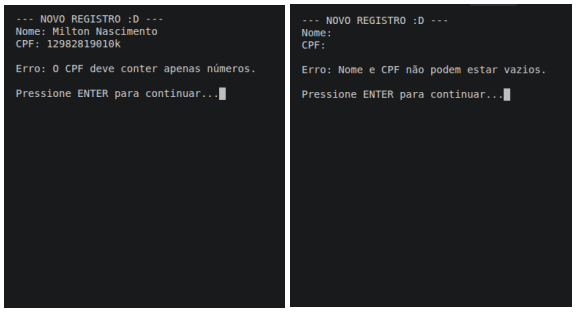


Fig. 3 e 4 - Exemplos de validação de campos vazios e de CPF com caracteres não numéricos.



Fig. 5 e 6 - Mensagens de permissões apresentadas após a autenticação de gerente e operador.

Os testes de autenticação demonstraram a identificação de funcionários de cargos distintos e a apresentação de mensagens de permissões específicas. No código, essa diferenciação é implementada por herança e pela sobrescrita de 'obter_permissoes()'. Além das mensagens, o menu de gestão aplica a restrição de reposição do estoque à gerência.

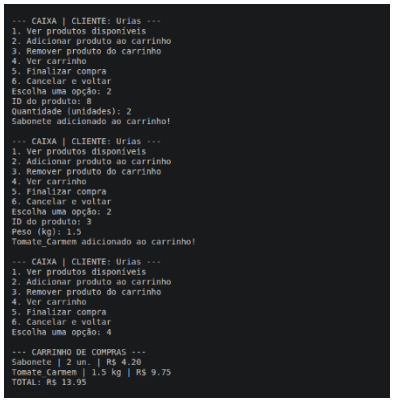


Fig. 7 - Adição de produtos unitários e pesáveis e cálculo do total do carrinho.

Na demonstração da Fig. 7, foram adicionadas duas unidades de sabonete, com subtotal de R$ 4,20, e 1,5 kg de tomate, com subtotal de R$ 9,75. O sistema calculou corretamente o total de R$ 13,95. O exemplo evidencia a interação entre 'Caixa', 'Estoque' e os objetos 'Produto', além do uso do método polimórfico de cálculo do valor de venda.

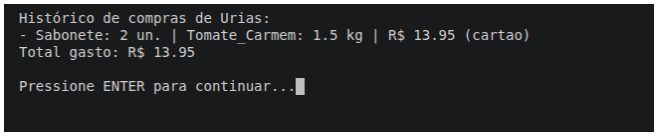


Fig. 8. Registro da compra no histórico do cliente após a finalização.

Após a finalização, a compra foi registrada no histórico do cliente, incluindo os produtos, as quantidades, o valor total e a forma de pagamento utilizada. O histórico apresentou o total gasto de R$ 13,95, correspondente ao valor exibido no carrinho. Esse resultado confirma a comunicação entre 'Caixa' e 'Cliente' e a gravação das informações da compra.

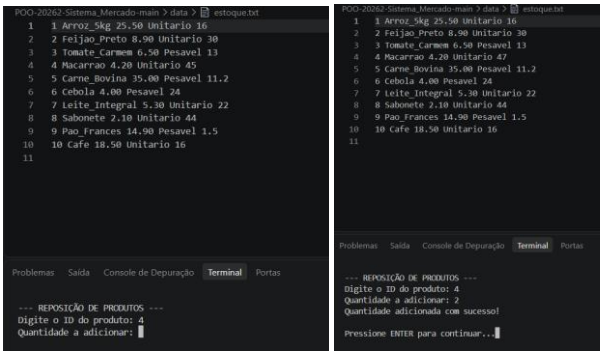


Fig. 9 e 10 - Arquivo de estoque antes e depois da reposição de duas unidades de macarrão.

Nas Fig. 9 e 10, o arquivo 'estoque.txt' registra inicialmente 45 unidades de macarrão. Após a reposição de duas unidades, o arquivo passa a registrar 47, e o terminal informa que a quantidade foi adicionada com sucesso. Como a alteração é gravada no arquivo, o resultado demonstra a persistência dos dados do estoque.

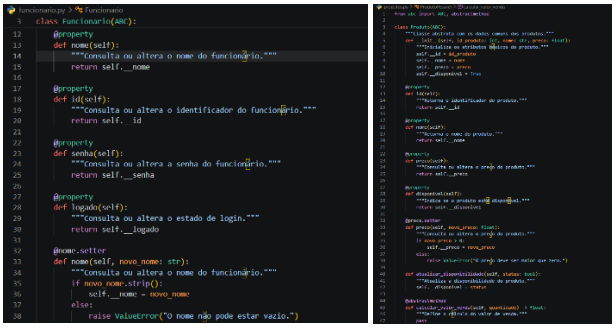


Fig. 11 e 12 - Trechos de código referentes à classe abstrata Produto e às propriedades de acesso.

Também foram analisados registros do código que mostram o uso de docstrings, atributos privados, propriedades e métodos abstratos. A árvore de diretórios e o histórico de commits documentam, respectivamente, a organização dos módulos e o acompanhamento das alterações durante o desenvolvimento. Os testes apresentados são demonstrações manuais; não foram incluídos testes automatizados no projeto entregue.

IV. CONCLUSÃO.

O desenvolvimento do sistema de gerenciamento de supermercado permitiu aplicar, na prática, os principais conceitos da Programação Orientada a Objetos estudados ao longo da disciplina. A implementação das funcionalidades de cadastro de clientes e funcionários, consulta e gerenciamento do estoque, realização de compras e registro do histórico de vendas possibilitou construir um sistema funcional, capaz de representar operações comuns de um supermercado.

A organização do código em classes com responsabilidades específicas contribuiu para a estruturação do projeto e para a interação entre seus diferentes componentes. A utilização de classes abstratas, como 'Produto' e 'Funcionario', permitiu estabelecer contratos para suas subclasses, enquanto a herança e o polimorfismo possibilitaram implementar comportamentos específicos para diferentes tipos de produtos e funcionários. Além disso, o encapsulamento, por meio de atributos privados e métodos de acesso, contribuiu para o controle das informações manipuladas pelo sistema. A persistência dos dados em arquivos de texto também permitiu manter informações de clientes, funcionários, estoque e histórico de compras entre diferentes execuções do programa.

Os resultados obtidos demonstraram que as funcionalidades implementadas operam conforme o comportamento esperado nos cenários apresentados durante a demonstração do sistema. O tratamento de exceções e a validação das entradas também contribuíram para evitar operações inválidas e tornar a interação mais consistente. Entretanto, o projeto apresenta algumas limitações, como a utilização de arquivos de texto em vez de um banco de dados, a interface restrita ao terminal e a simulação dos métodos de pagamento, sem integração com serviços bancários ou operadoras de cartão.

Dessa forma, conclui-se que os objetivos do projeto foram alcançados dentro do escopo proposto, permitindo desenvolver um sistema que integra diferentes operações de gerenciamento de supermercado e demonstra a aplicação dos conceitos fundamentais de Programação Orientada a Objetos.

REFERÊNCIAS

[1] "Self-checkout: a sofisticação da tecnologia cada vez mais presente nos supermercados," ABRAS, Clipping Tecnologia. [Online]. Disponível em: https://www.abras.com.br/clipping/tecnologia/115840/self-checkout-a-sofisticacao-da-tecnologia-cada-vez-mais-presente-nos-supermercados. Acesso em: 10 nov. 2026.

[2] B. B. et al., "POO-20262-Sistema_Mercado," GitHub, 2026. [Online]. Disponível em: https://github.com/raflarii/POO-20262-Sistema_Mercado.git. Acesso em: 10 nov. 2026.

[3] B. B. et. Al, "POO-20262-Sistema_Mercado," Diagrama UML disponível na pasta 'docs' no GitHub, 2026. [Online]. Disponível em: https://github.com/raflarii/POO-20262-Sistema_Mercado.git. Acesso em: 10 nov. 2026.